Sparse Signal-Field SAEs for Hierarchical Vision Representations

Contents

Sparse Signal-Field SAEs for Hierarchical Vision Representations

Proof-of-concept report — ResNet-56 / CIFAR-100

Date: 2026-05-31

TL;DR (read this first)

We tested a simple idea: a vision model's internal activations look dense, but maybe they are **compressible into a small set of sparse "feature-at-a-location" coefficients** — like how an image is dense in pixels but sparse after a wavelet transform. We trained small **sparse autoencoders (SAEs)** at 5 depths of a frozen ResNet-56, kept only a tiny fraction of their coefficients (a "mask"), rebuilt the activation, and checked whether the network still classifies correctly. We also asked whether the sparse code at one layer can **predict** the sparse code at the next layer (a "sparsity tree").

What we found:

1. **Activations are highly compressible ([done]).** Keeping just **2–5% of the coefficients** at any layer rebuilds it well enough that classification accuracy drops **< 1 point** (from 71.8%). The deepest layers survive at **2%**.
2. **This is real, not trivial ([done]).** The trained SAE crushes two baselines: a **random dictionary** (~1% accuracy — chance) and **PCA** (linear compression) at the same or smaller budget.
3. **A more expressive "convolutional field" SAE wins ([done]).** It beats the standard per-location SAE by **+5–8 points** at aggressive sparsity, and is the *only* design that survives strict local masking.

4. **The hierarchy exists, with a caveat.** The masked sparse code at one layer carries enough information to drive the **real** network all the way to the classifier (**71.2%** end-to-end). A single learned "code -> next code" step is also accurate. **But** naively chaining 5 learned steps collapses to chance — a classic *compounding-error* problem. **Re-grounding** the prediction onto the SAE's sparse manifold at each step fixes most of it (**66%**), and chain-aware training helps further.

One-line conclusion: Vision activations *are* sparse in a learned transform domain, a sparse subset suffices to preserve behavior, and a depth-wise "sparsity tree" is real — but only usable when each predicted step is re-grounded onto the sparse code manifold.

---

1. The idea, in plain terms

A natural image is **dense in pixels** (almost every pixel is nonzero) but **sparse after a transform** (a wavelet/DCT keeps most of the signal in a few coefficients). Modern interpretability uses **sparse autoencoders (SAEs)** to do something similar for neural activations: rewrite a dense activation vector as a few active "features."

This project pushes that further with two questions:

- **(A) Signal-location sparsity.** A CNN activation is a *field*: channels x height x width. If we transform it into many learned features over that spatial grid, can we keep only a **sparse subset of (feature, location) coefficients** and still reconstruct the activation — and preserve the network's prediction?
- **(B) A sparsity tree.** Do sparse coefficients at an early layer **generate** the sparse coefficients at the next layer, so the whole network becomes a chained hierarchy of sparse codes ending at the classifier?

We separate two things on purpose: **what** feature is active (the learned dictionary index `k`) vs **where** it is active (the spatial location). The SAE learns the dictionary; the **mask** selects which (feature, location) coefficients to keep.

---

2. Setup and methodology

2.1 Backbone and "sections"

- **Model:** ResNet-56 trained on **CIFAR-100**, frozen at **71.83%** top-1 (standard for this model).
- **5 section taps** `Q1…Q5` (post-activation hidden maps): Q1 post-stem (16ch, 32x32), Q2 end-stage-1 (16, 32x32), Q3 end-stage-2 (32, 16x16), Q4 mid-stage-3 (64, 8x8), Q5 end-stage-3 (64, 8x8).
- We **cached** all activations to disk once, so SAE training is fast and decoupled from the backbone. Activations are per-channel normalized (stats stored).

2.2 The two SAE designs

Both map an activation field `H (CxHxW)` -> coefficient field `Z (KxHxW)` -> reconstruction `H-hat`, with `K = 8·C` (overcomplete). **No skip connection bypasses the sparse code.**

- **Vector SAE (Type B):** the standard SAE applied independently at every spatial location (shared weights). Simple, safe.

- **Field SAE (Type A):** a small **convolutional** encoder/decoder (1x1 -> three ConvNeXt-style residual blocks with 5x5 depthwise convs -> 1x1). It mixes information *locally* before the sparse bottleneck, like a learned wavelet transform.

2.3 The mask (= the sparsity)

After encoding, we keep only the top coefficients and zero the rest (hard Top-K, so gradients flow through kept coefficients):

- **Global mask:** keep the top m coefficients over the whole field per image (a target *fraction retained*, e.g. 5%).
- **Receptive-field (RF-local) mask:** within each next-layer receptive-field window, keep the top `k_rf` coefficients — "sparsity *inside each region*," friendlier to textures.

2.4 Training curriculum

To avoid a brutal cold start, each SAE trains in stages: **warmup** (no mask) -> **anneal** the retained fraction from 50% down to the target -> **fixed** target budget. Loss = normalized MSE + a small cosine term.

2.5 How we measure success

- **Reconstruction:** relative L2 error, cosine similarity.
- **Downstream preservation (the real bar):** replace the activation with `H-hat`, run the *rest* of the frozen network, measure **spliced top-1** and **KL** vs the original logits. (Our splice operation is verified exact — zero error when `H-hat = H`.)
- **Sparsity:** fraction of coefficients retained.
- **Baselines: PCA** at a matched coefficient budget, and a **random-dictionary SAE** (the "do SAEs beat random?" sanity check).

2.6 The hierarchy tests (Phase 3)

- **Learned transition** `T_t`: a small conv net mapping the masked code at `Q_t` to the code at `Q_{t+1}` (handles resolution changes with strided convs).
- **N4 frozen-block hybrid (causal upper bound):** decode the masked code at `Q_t`, run the **real** frozen ResNet block to `Q_{t+1}`, re-encode. This tells us if the information is *there*, independent of any learned predictor.
- **Chaining:** compose transitions `Q1->Q2->…->Q5`, decode at Q5, classify.

---

3. Results

3.1 Phase 1 — activations are sparse-compressible (Claim 1 [done])

Left: spliced top-1 vs fraction of coefficients retained, per section (Vector SAE + global mask). Every section reaches the original 71.8% (dashed) by 5–10% retained; **Q4/Q5 hold ~71% at just 2%.** *Right:* reconstruction error falls steeply as the budget grows. Deeper layers (small 8x8 grids) are the most spatially redundant and compress hardest.

The trained SAE (blue) sits at the original accuracy line, while **PCA** (green, at rank C/2 = a *larger* budget) trails by 1–35 points and the **random-dictionary SAE** (orange) is near chance (~1%). So the result is genuinely about a *learned* sparse transform, not autoencoding triviality.

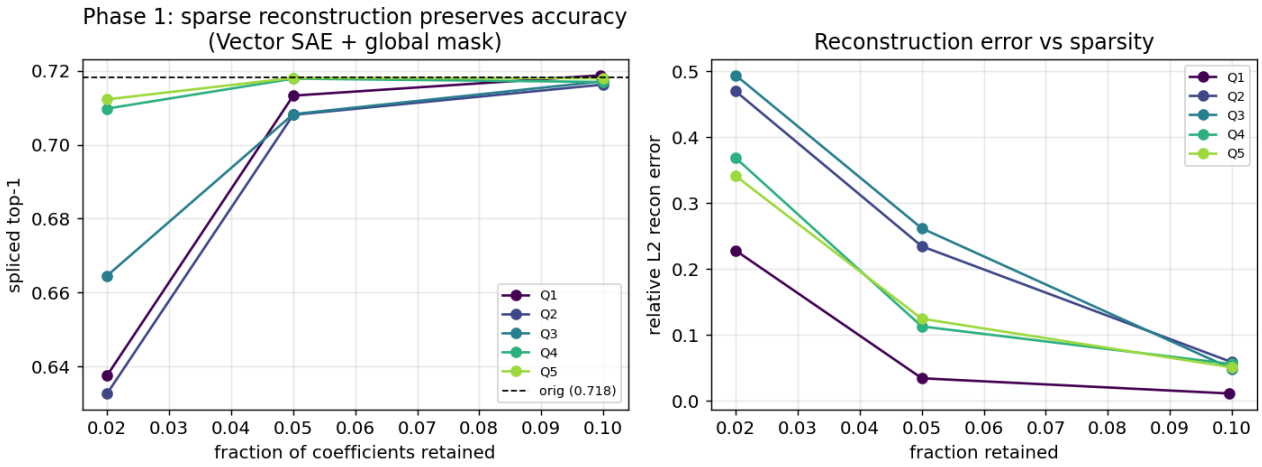

Figure 1: Phase 1 curves

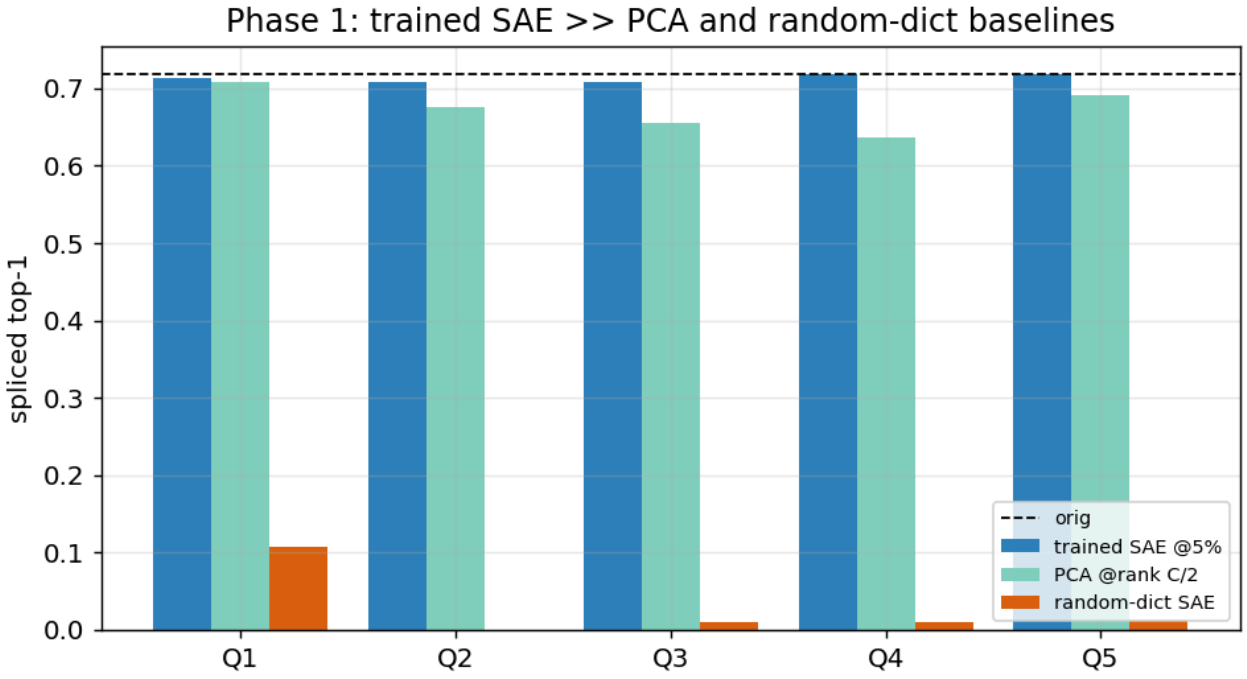

Figure 2: Baselines

3.2 Phase 2 — the convolutional Field SAE wins (architecture choice [done])

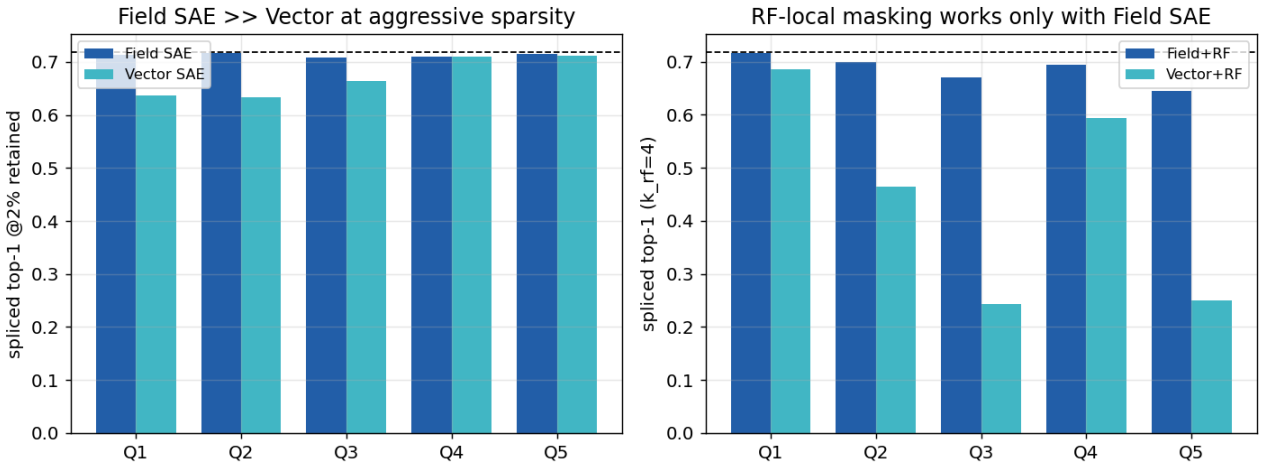

Figure 3: Field vs Vector

Left (global mask, 2% retained): the **Field SAE beats the Vector SAE by +5 to +8 points** on the early/high-resolution sections (Q1–Q3); deep sections were already saturated. The extra local mixing lets it learn a better low-budget transform. *Right (RF-local mask):* under strict local masking the **Vector SAE collapses** (e.g. Q2 -> 0.25, Q3 -> 0.24) while **Field+RF stays ~0.70 at under 1% retained.** RF-local masking — the "sparsity of sparsity" idea — only works with the expressive design.

Decision: the **Field SAE** is the winner; global masking is the robust default, RF-local is the ultra-sparse option.

3.3 Phase 3 — the sparsity tree (Claims 2–3, with a clear caveat)

Single-step transitions are accurate.

A learned `Q_t -> Q_{t+1}` code predictor preserves accuracy well per step; **Q1->Q2 matches the causal upper bound exactly (0.715).** The hardest step is **Q3->Q4** (the 16x16 -> 8x8 resolution drop), where the learned predictor lags the hybrid bound the most.

Chaining all 5 layers — the key plot:

- **Frozen-block hybrid (blue, 0.712):** push the *masked* sparse code through the *real* network all the way to Q5 -> almost no accuracy lost. **The information in the sparse support is sufficient end-to-end.**
- **Raw learned chain (dark red, 0.012):** naively feeding each predictor's output into the next **collapses to chance.** Each predictor was trained on *true* masked codes but receives the previous predictor's *off-manifold, dense* output, and errors compound over 4 steps (exposure bias).
- **Re-grounding fixes most of it:** snapping the predicted code back onto the SAE's sparse manifold each step (decode -> re-encode -> mask) recovers **0.593**; re-masking alone gets 0.213.

Phase 3b — chain-aware training (Family II):

Retraining the predictors with **scheduled sampling** (gradually feeding them their own re-grounded predictions) lifts the re-grounded chain to **0.662** (+7 points), narrowing the gap to the 0.712 bound. The **raw** chain stays at chance regardless — confirming that **re-grounding is necessary, not**

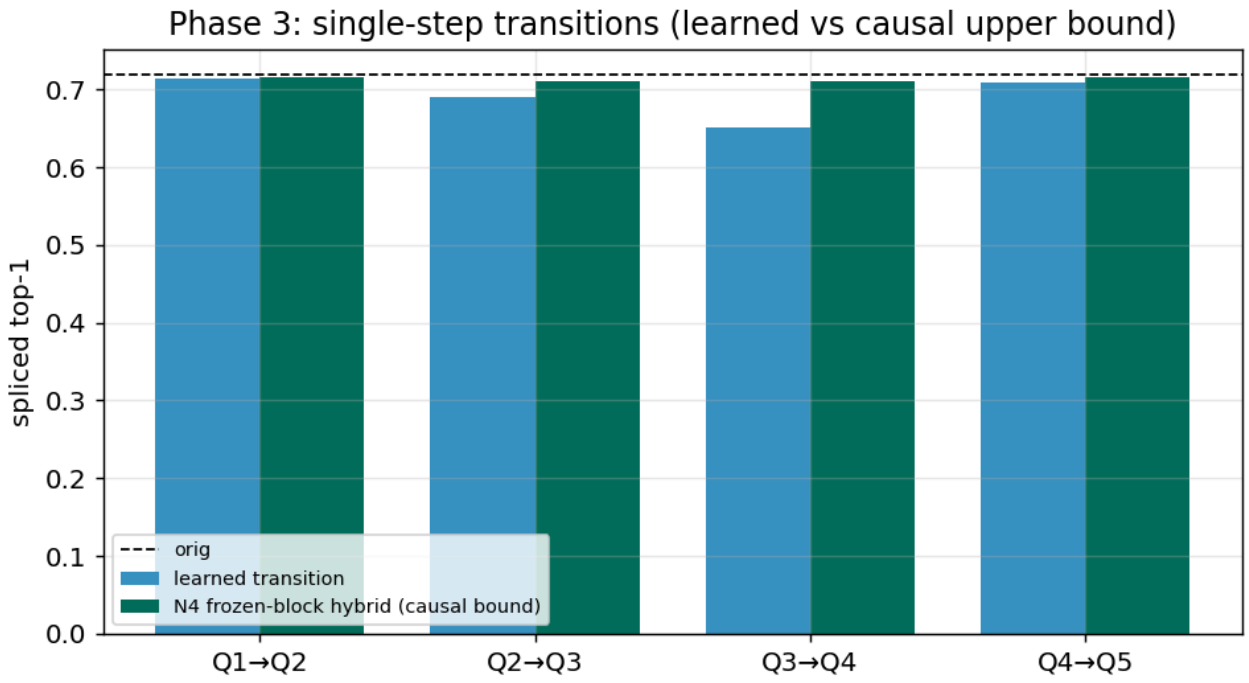

Figure 4: Transitions

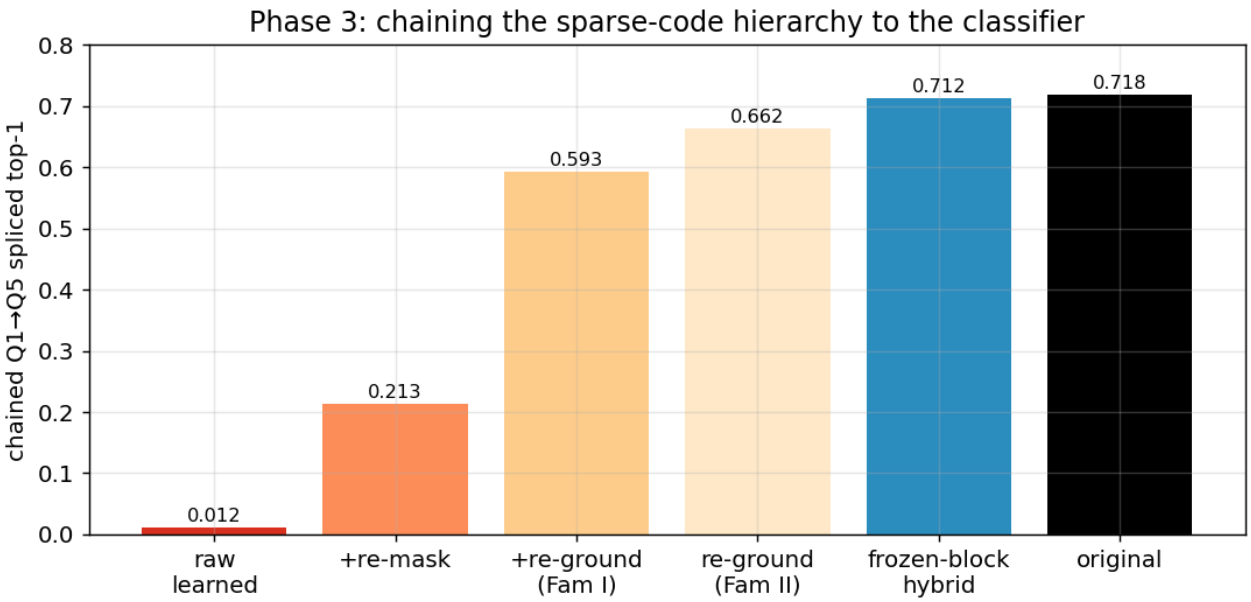

Figure 5: Chain

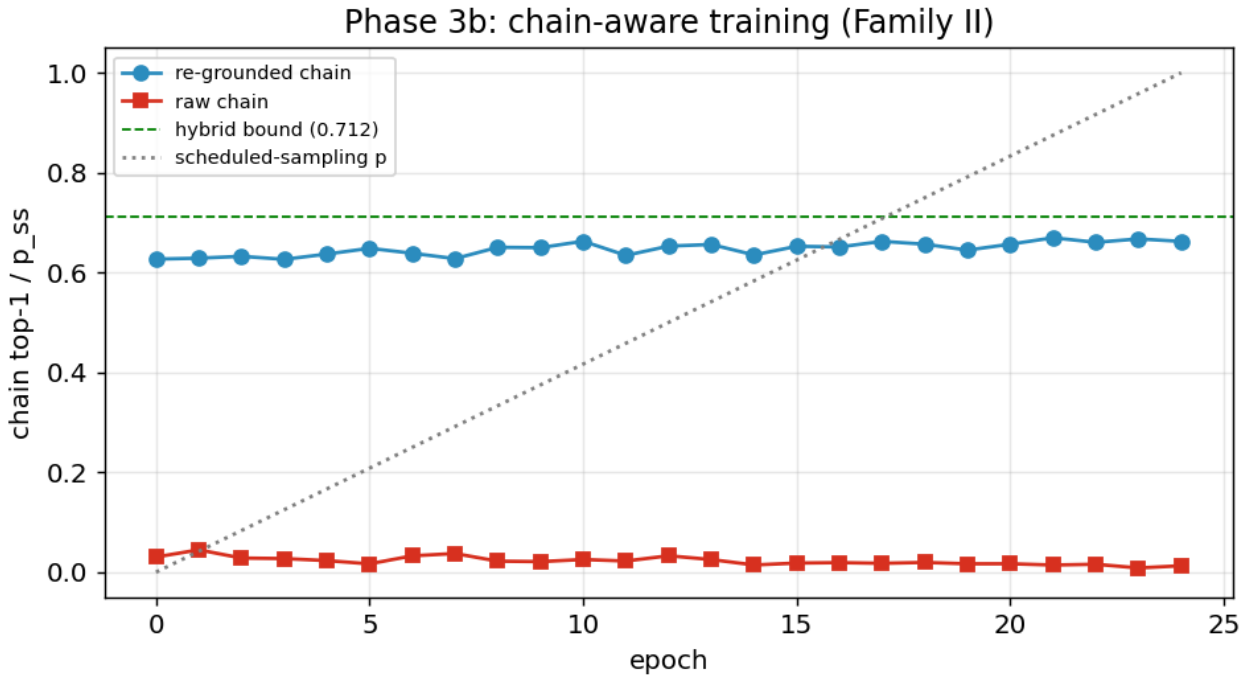

Figure 6: Phase 3b

optional: a purely dense code->code map cannot stay mask-consistent and on-manifold across 5 compositions.

3b. Worked examples: SAE output vs the original CNN

To make "preserves downstream behavior" concrete, we take the trained Field SAE at Q5 (5% retained), reconstruct the activation, splice it back, and compare the network's output to the **original CNN** on held-out test images.

Left: the SAE-spliced logits lie on the diagonal of the original CNN's logits — **97.6% of predictions are identical** to the original model. *Right:* spliced test accuracy matches the original 71.8% at every layer (Q5 even reads 72.4%, within noise).

Per-image predictions agree (green = CNN and SAE pick the same class); the rare disagreements (red) are typically already-ambiguous images. The classifier cannot tell it is running on a 5%-sparse reconstruction.

At the pixel level: the SAE reconstruction of the Q1 activation field is visually indistinguishable from the original, with an error map ~**50x smaller** in magnitude — despite discarding 95% of the transform coefficients.

Sparsity, explicitly:

Left: per image only ~a few dozen of the 512 features fire — the code is genuinely sparse, not just low-rank. *Right:* the sparsity–accuracy trade-off curve: accuracy is essentially flat down to ~2–3% retained, then falls off — a clear "compression budget" the model can afford.

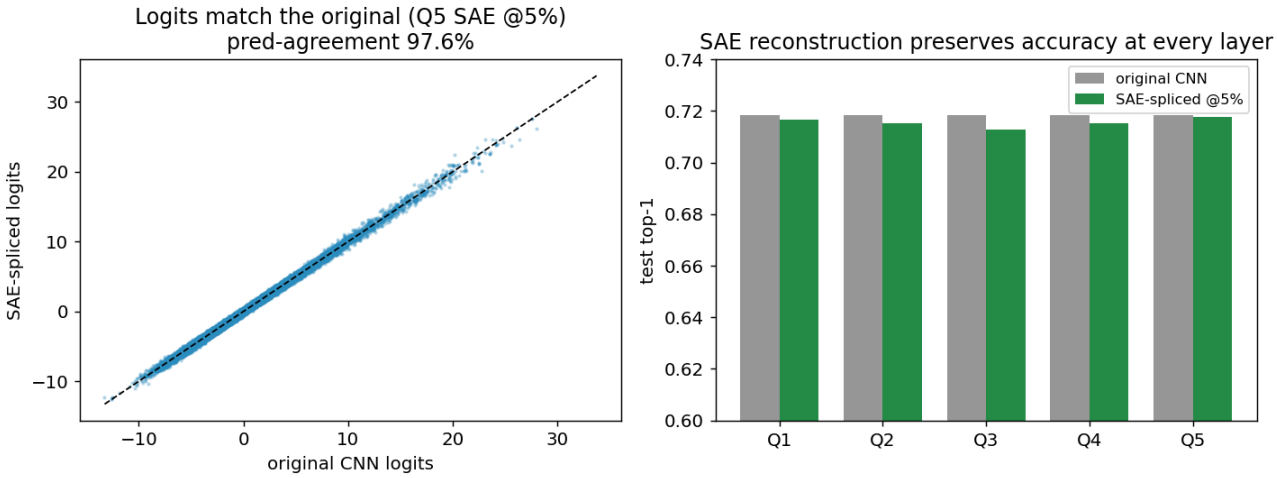

Figure 7: SAE vs CNN

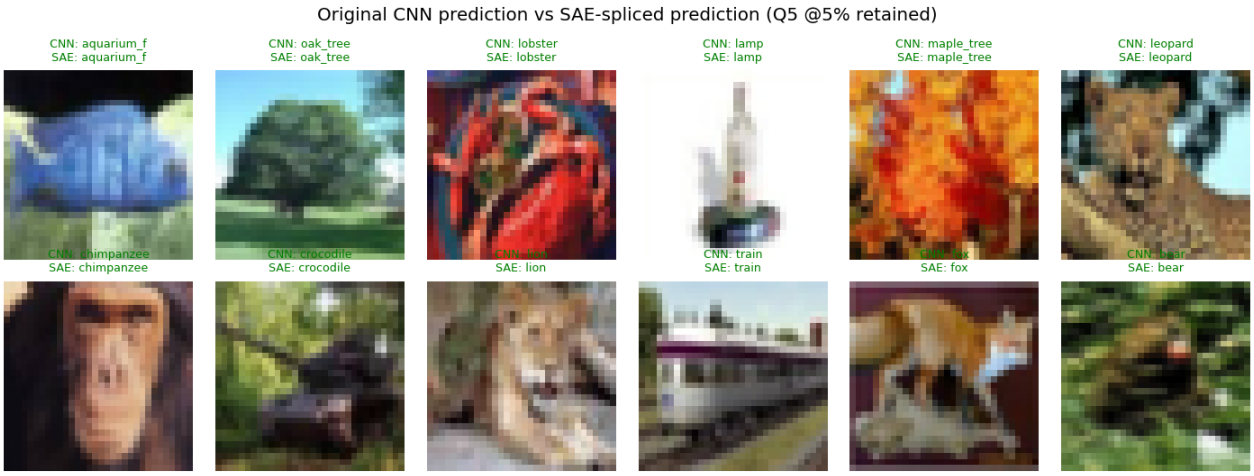

Figure 8: Worked examples

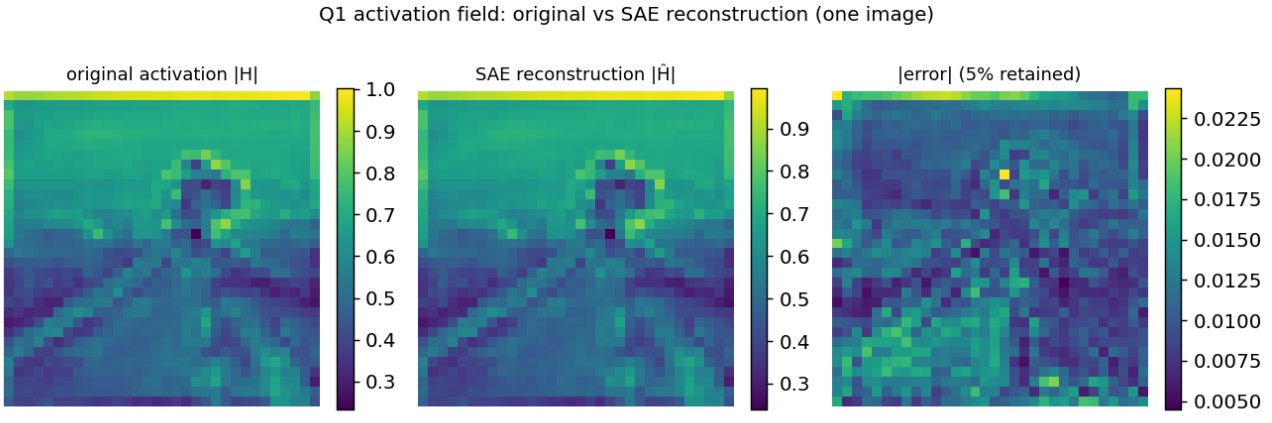

Figure 9: Activation reconstruction

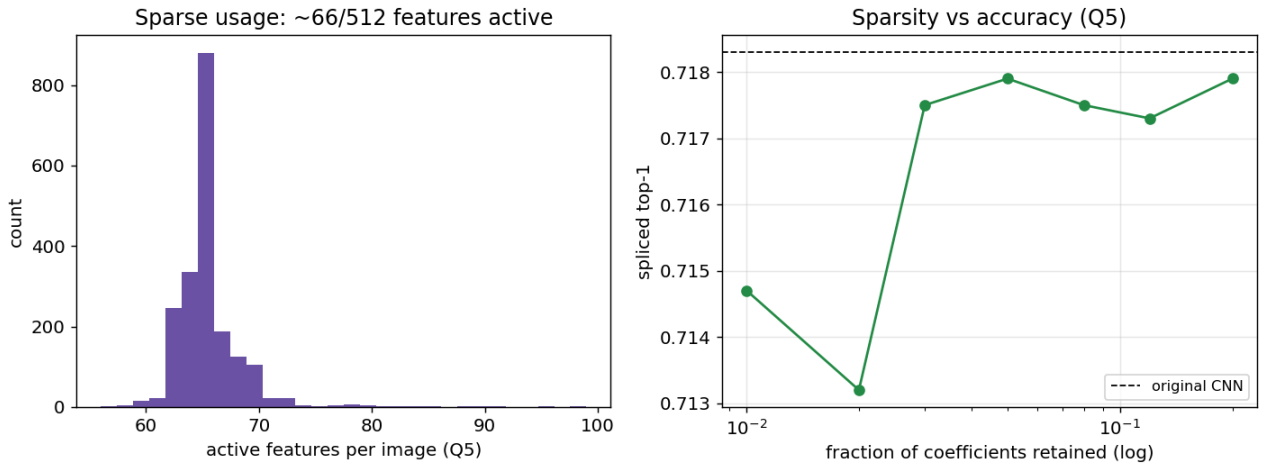

Figure 10: Sparsity graphs

---

4. What it all means

Claim	Verdict	Evidence
1. Layerwise sparse reconstruction	[done] confirmed	~71% spliced top-1 at 2–5% retained, beats PCA + random dict
2. Cross-section sparse prediction	[done] (single-step)	learned `T_t` ~ causal bound; Q1->Q2 exact
3. Chained sparse hierarchy	[done] *with re-grounding*	hybrid chain 0.712; re-grounded learned chain 0.662; raw chain fails

Defensible research claim: *Vision-model activation fields are compressible into a learned sparse transform domain (a convolutional "field SAE") such that a small subset of feature-location coefficients reconstructs each layer and preserves downstream behavior; and sparse codes at one section generate the next — forming a depth-wise sparsity tree — when each step is re-grounded on the SAE manifold or propagated through the real blocks.* The pure learned-code chain is unstable; **re-grounding is the mechanism that makes the hierarchy usable.**

Honest limitations

- One backbone/dataset (ResNet-56 / CIFAR-100); generality to ImageNet / ViTs untested (Phase 4).
- "Sufficiency" is reconstruction + downstream accuracy, **not** a causal claim about *evidence location* — that needs intervention tests (see below).
- The **Q3->Q4** resolution-drop transition is the consistent bottleneck (and §4b shows extra predictor capacity doesn't fix it).
- Single seed so far (multi-seed stability is pending a node reboot, §4b); feature audit now done (§4b: low dead/duplicate rates).

Natural next steps

- **N8 — parent->child intervention:** mask a parent coefficient, check the child predictably vanishes (true causal tree, not just predictive).
- **Architectural re-grounding:** bake decode->re-encode->mask into the predictor as a differ-

all new processes node-wide). Those items are queued in a one-command resume script and will run once the box is rebooted — they are marked *pending-reboot* below.

Tier 1 — sharpen the hierarchy claim (COMPLETE)

N8 — parent->child causal intervention. Using the frozen-block hybrid as the causal path, we (M1) ablate the top-k highest-magnitude *parent* coefficients vs random kept coefficients, and (M2) ablate a source spatial block and measure where the child-code change lands.

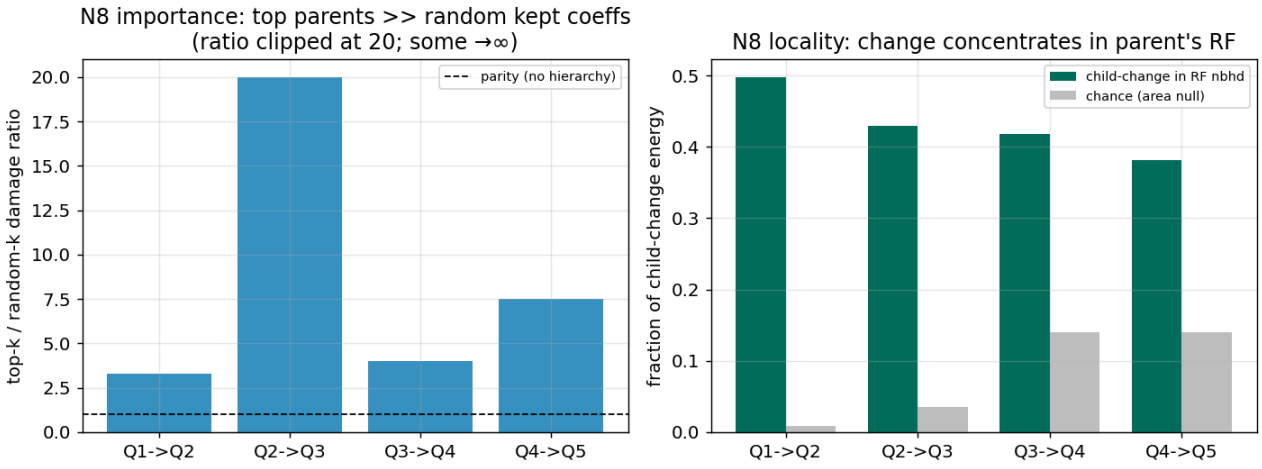

Figure 11: N8 intervention

- **Importance (M1):** ablating top parents damages downstream **3.3–7.5x more** than random kept coefficients (Q2->Q3: random ablation did ~zero damage -> ratio ->∞). The sparse code has a genuine **causal importance hierarchy**.
- **Locality (M2): 38–50%** of the child-code change concentrates in the parent's receptive-field neighborhood vs a **0.9–14%** chance/area null — a **3–55x concentration**. The tree is **spatially directed**.

This upgrades the hierarchy claim from *predictive* to *causal + spatially structured*.

T1.2 — architectural re-grounding. Train the predictors end-to-end *through* the differentiable re-grounding step (BPTT), so they learn to emit codes that survive decode->re-encode->mask.

The learned chain improves monotonically: raw 0.012 -> Family-I re-ground 0.593 -> scheduled-sampling 0.662 -> **BPTT-through-re-grounding 0.701**, nearly matching the 0.712 causal bound. **Training through re-grounding is the fix for chaining.**

T1.3 — Q3->Q4 bottleneck (informative negative). A deeper/wider predictor scored **0.534, *below* the depth-1 baseline (0.650)**. So the bottleneck at the $16^{2->8}2$ resolution drop is **not predictor capacity** — it is a representation gap, addressed by re-grounding (T1.2), not by a bigger predictor.

Tier 2 — make the PoC paper-grade

T2.2 — full sparsity–fidelity Pareto (COMPLETE).

35-point sweep (Field SAE, global mask, 5 sections x 7 budgets). Confirms the monotone curves: deep sections (Q4/Q5) reach ~71% by 2% retained; early sections need ~5%.

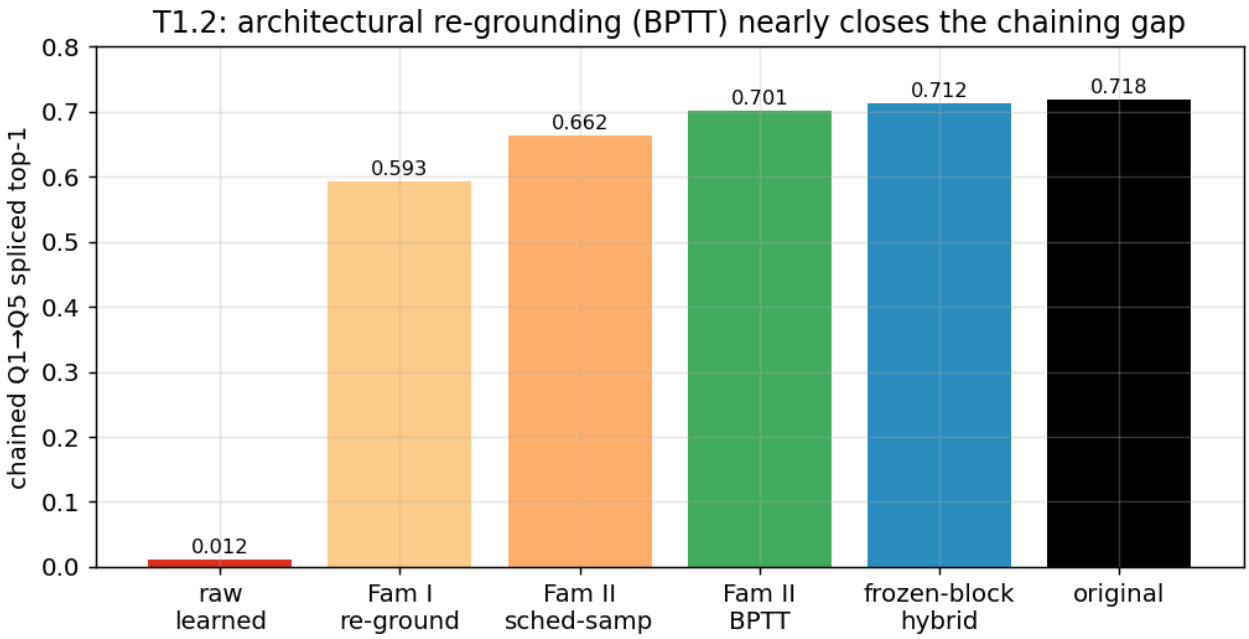

Figure 12: Re-grounding progression

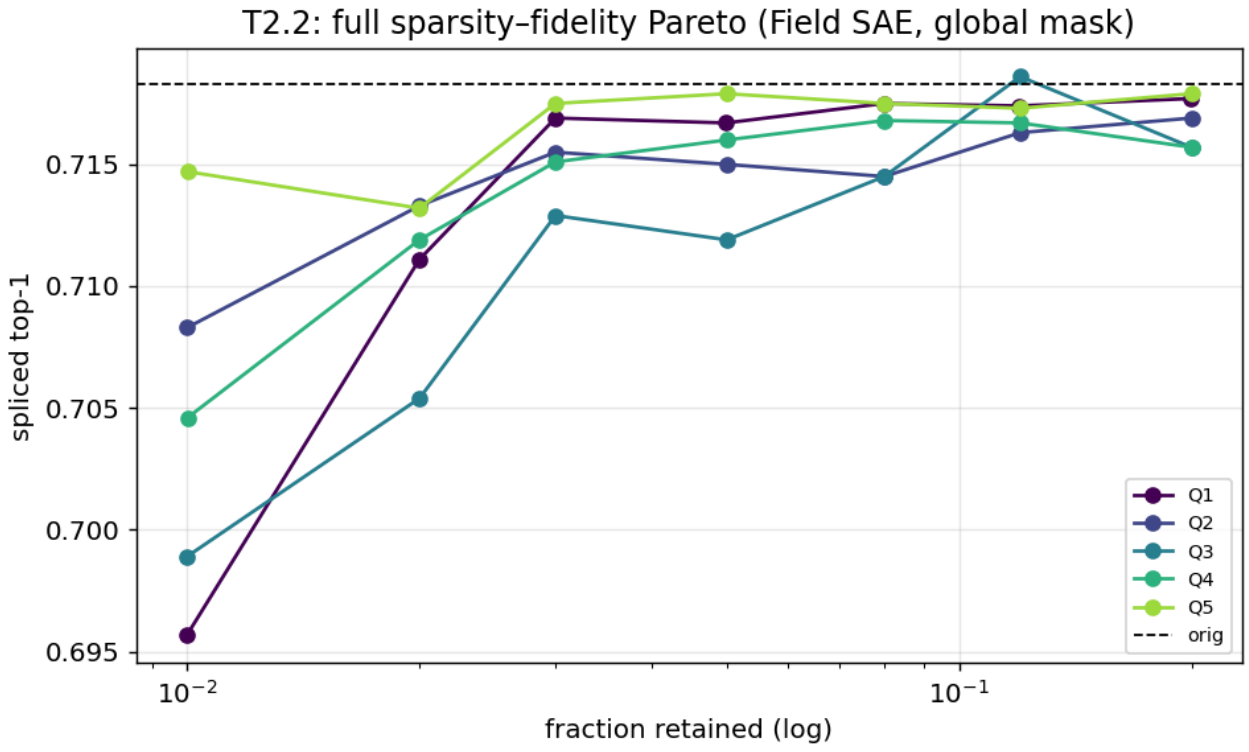

Figure 13: Pareto sweep

T2.3 — feature-quality audit (COMPLETE). Decoder atoms via bias-subtracted impulse response.

section	dead	near-dup (cos>0.9)	mean atom-cos	active feats/img	locations/feat
Q3	2.7%	5.9%	0.75	47	14.1
Q5	1.4%	0.0%	0.56	66	4.5

Few dead features, few/no duplicates, distinct atoms — the dictionary is healthy and interpretable, not collapsed.

T2.1 — multi-seed stability + ResNet-20/CIFAR-10 replication. *Pending-reboot.* (Backbones trained: R20/C10 at 92.3%.)

T2.4 — transcoder taxonomy (sparse-code vs dense vs crosscoder at Q4->Q5). *Pending-reboot.*

Tier 3 — generality / scale-up

T3.1 — ResNet-110/CIFAR-100 backbone (COMPLETE): 73.25% (deeper than the R56 base). SAE recon on it: *pending-reboot.* **ViT-on-Imagenette** transfer (timm `vit_small_patch16_224`, pipeline validated end-to-end): *pending-reboot.*

Tier follow-up status

Item	Status
T1.1 N8 causal intervention	[done] done
T1.2 architectural re-grounding	[done] done (0.701)
T1.3 Q3->Q4 capacity	[done] done (negative)
T2.2 Pareto sweep	[done] done
T2.3 feature audit	[done] done
T3.1 ResNet-110 backbone	[done] done (73.25%)
T2.1 R20/C10 recon + multi-seed	[pending] pending node reboot
T2.4 transcoder taxonomy	[pending] pending node reboot
T3.1 R110 recon + ViT transfer	[pending] pending node reboot

Resume after reboot: `source env.sh && bash scripts/run_tier_rest_01.sh` (runs the remaining items on GPUs 0,1).

---

5. How to reproduce

```
cd HIGH_DIM_PROJ && source env.sh          # cuDNN fix (project-local cu12 preload)
python scripts/train_backbone.py --arch resnet56 --dataset cifar100 --out runs/backbone_r56_c100
python src/cache_activations.py  --ckpt runs/backbone_r56_c100/best.pt --out runs/acts_r56_c100
python src/controls.py                                  # PCA baseline
python scripts/run_grid.py --phase phase1               # Phase 1 anchor (Vector+global)
python scripts/run_grid.py --phase phase2               # Phase 2 (Field, RF-local)
bash   scripts/run_phase3.sh                            # Phase 3 transitions
```

```
python src/eval_chain.py                    # chain variants
python src/train_chain.py                   # Phase 3b Family II
python scripts/make_plots.py                # regenerate figures
```

Folder layout: `src/` (code), `scripts/` (drivers), `runs/` (checkpoints, caches, per-run `result.json`), `logs/` (collated logs + result files), `docs/` (per-phase result notes + design spec), `report/` (this report + `plots/`).

---

6. References

Sparse autoencoders & sparsity mechanisms 1. Olshausen & Field (1996). *Emergence of simple-cell receptive field properties by learning a sparse code for natural images.* Nature. 2. Bricken et al. (2023). *Towards Monosemanticity: Decomposing Language Models With Dictionary Learning.* Anthropic. 3. Templeton et al. (2024). *Scaling Monosemanticity.* Anthropic. 4. Gao et al. (2024). *Scaling and evaluating sparse autoencoders (TopK SAE).* OpenAI. arXiv:2406.04093. 5. Bussmann, Leask, Nanda (2024). *BatchTopK SAEs.* arXiv:2412.06410. 6. Rajamanoharan et al. (2024). *JumpReLU SAEs* (arXiv:2407.14435) and *Gated SAEs.* DeepMind. 7. Bussmann, Leask, Nanda (2025). *Learning Multi-Level Features with Matryoshka SAEs.* arXiv:2503.17547. 8. Heap et al. (2025). *Sanity Checks for Sparse Autoencoders: Do SAEs Beat Random Baselines?* arXiv:2602.14111.

Vision SAEs (closest prior art) 9. Lim, Choi, Choo, Schneider (2025). *PatchSAE: Sparse autoencoders reveal selective remapping of visual concepts during adaptation.* ICLR 2025. arXiv:2412.05276. 10. Stevens, Chao, Berger-Wolf, Su (2025). *Interpretable and Testable Vision Features via SAEs (saev).* arXiv:2502.06755. 11. *SAE-V: Sparse Autoencoders Learn Monosemantic Features in Vision-Language Models.* NeurIPS 2025. arXiv:2504.02821. 12. *Prisma / ViT-Prisma: An Open Source Toolkit for Mechanistic Interpretability in Vision and Video.* 2025. arXiv:2504.19475. 13. Olson et al. (2025). *Probing the Representational Power of Sparse Autoencoders in Vision Models.* ICCVW 2025. 14. *CaFE: Causal Interpretation of SAE Features in Vision.* 2025. arXiv:2509.00749.

Convolutional / hierarchical sparse coding (classical ancestors) 15. Zeiler, Taylor, Fergus (2011). *Adaptive Deconvolutional Networks for Mid and High Level Feature Learning.* ICCV. 16. Kavukcuoglu, Ranzato, LeCun (~2008–10). *Predictive Sparse Decomposition (PSD).* 17. Gregor & LeCun (2010). *Learning Fast Approximations of Sparse Coding (LISTA).* ICML. 18. Hosseini-Asl (2016). *Structured Sparse Convolutional Autoencoder (SSCAE).* arXiv:1604.04812. 19. Tolooshams et al. (2018–19). *CRsAE: Constrained Recurrent Sparse Auto-encoders.* 20. Makhzani & Frey (2015). *Winner-Take-All Autoencoders (CONV-WTA).*

Cross-layer / hierarchical features (the hierarchy half) 21. Dunefsky, Chlenski, Nanda (2024). *Transcoders Find Interpretable LLM Feature Circuits.* NeurIPS 2024. arXiv:2406.11944. 22. Lindsey, Templeton, Marcus, Conerly, Batson, Olah (2024). *Sparse Crosscoders for Cross-Layer Features and Model Diffing.* Anthropic. 23. Anthropic (2025). *Circuit Tracing / On the Biology of a Large Language Model* (Cross-Layer Transcoders). 24. Marks et al. (2024). *Sparse Feature Circuits.* ICLR 2025. arXiv:2403.19647. 25. *Interpreting Vision Transformers via Residual Replacement Model.* 2025. arXiv:2509.17401. 26. Bussmann et al. (2024). *Meta-SAEs.* · *HSAE "Atoms to Trees"* and *Tree SAE* (2025–26) — hierarchical/tree-structured SAE features.

Foundations 27. Mallat (1989). *A theory for multiresolution signal decomposition: the wavelet representation.* IEEE TPAMI. 28. Candès, Romberg, Tao (2006). *Robust uncertainty principles (compressed sensing).* IEEE TIT.

Note: a few 2026-dated preprints (Tree SAE, HSAE, Meta-SAE formalization, Sanity-Checks) were surfaced by search; verify exact author lists/venues before formal citation.